

Confidential Computing

Introduction & Confidential Computing Foundations

Pr. Valerio Schiavoni

University of Neuchâtel

Valerio.Schiavoni@unine.ch

Tentative Agenda of the Course (v.1)

1. Introduction & Confidential Computing Foundations

2. Hardware Root of Trust

3. TEE Fundamentals

4. Intel SGX Deep Dive

5. AMD SEV, SEV-ES, SEV-SNP

6. Intel TDX

7. Confidential Containers

This schedule
can change!

Some chapters
can be merged
or removed.

Agenda

- Motivation: Why do we need Confidential Computing?
- The Data Lifecycle:
 - data-at-rest, in-transit, and in-use
- What is Confidential Computing?
 - Key concepts and goals
- Trusted Execution Environments (TEEs)
- Threat models: Who are we protecting against?

Lecture objectives

- Understand the principles, architectures, and threat models behind confidential computing systems
- Analyze and compare trusted execution environments (TEEs) such as Intel SGX, AMD SEV, ARM TrustZone, and Intel TDX
- Explore real-world applications, attacks, and mitigations in modern cloud and edge computing environments

Course Outcomes

- Identify and describe **key components** of confidential computing architectures and their security properties
- Evaluate the **security** and **performance trade-offs** of different TEE technologies and deployment models
- Apply learned concepts to assess threats and design secure applications using **confidential computing principles**

Section I

Motivation:

Cloud Threats and the Data Lifecycle

The Concept of Trust in Computing



The Concept of Trust in Computing

- **Trust: assumption of correct behavior without direct verification.**
- In computer systems, trust means believing that **a component will act as expected**, even if we cannot observe or prove it every time.

Layer	What we trust	Example
Hardware	Executes code faithfully	CPU, firmware
OS/Hypervisor	Manages memory & processes honestly	Linux, VMware
Cloud provider	Protects data & resources	AWS, Azure
Application	Handles secrets correctly	Banking app

Trust: Why It Matters

- Every trusted layer becomes part of your Trusted Computing Base (**TCB**).
- If one layer fails, all upper layers' guarantees collapse.
- Confidential Computing reduces this TCB:
 - by minimizing what must be trusted.

Trust is a vulnerability! Confidential Computing is to replace **blind trust** with **verifiable trust**.

Trust Assumptions in the Cloud

- Implicit trust in the cloud provider
 - Users assume the provider won't inspect, tamper with, or leak their data or workloads
- Trust in software stack integrity
 - OS, hypervisor, and management tools are expected to behave correctly and securely
- Why this trust exists
 - **Reputation** and compliance (e.g., ISO 27001)
 - Lack of alternatives for fine-grained hardware isolation
 - Convenience and cost-efficiency of cloud deployment

Everyone uses X-Cloud, why not?



Insider Threats in the Cloud

- Privileged access risk
 - Cloud admins often have access to virtual machines, storage, and logs — including sensitive customer data
- Realistic attackers
 - Malicious or compromised employees can exploit their access for espionage, financial gain, or sabotage
- Example scenarios:
 - Unauthorized snapshot of a VM or disk
 - Injection of debugging agents or backdoors
 - Abuse of internal monitoring tools

A Tesla Employee Thwarted an Alleged Ransomware Plot

Elon Musk confirmed Thursday night that a ransomware gang had approached a Gigafactory employee



Russian Indicted in Tesla Ransom Scheme

Faces Five Years in Prison If Convicted in Connection With Thwarted Scam

Prajeet Nair (@prajeetspeaks) • September 8, 2020



Credit Eligible

Get Permission



Tesla's Gigafactory in Nevada

A federal grand jury has formally indicted a Russian national in connection with a thwarted attempt at stealing corporate data from electric vehicle manufacturer Tesla so it could be used to extort a \$4 million ransom, according to the [U.S. Justice Department](#)

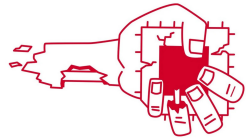
Example: Tesla Insider (2020)

- Background
 - A Tesla employee alerted the company to a malicious insider attempting to *exfiltrate* sensitive data and install malware on Tesla's internal systems.
- Insider Vector
 - The attacker (a Russian national) tried to bribe an employee \$1 million to plant ransomware on Tesla's Gigafactory systems.
- Implications
 - Highlights the real risk of internal compromise even in highly secured, tech-savvy environments — relevant to cloud operators.

External Attacks

- Side-Channel Attacks

- Exploit shared hardware (e.g., CPU caches, branch predictors) to infer sensitive data across VM boundaries
- Examples: Spectre, Meltdown, ZombieLoad



- Hypervisor escapes

- Attackers exploit vulnerabilities to break VM isolation and gain control of the host or other tenants
- Examples: VENOM (QEMU), Cloudburst (VMware)



- Physical theft or tapping

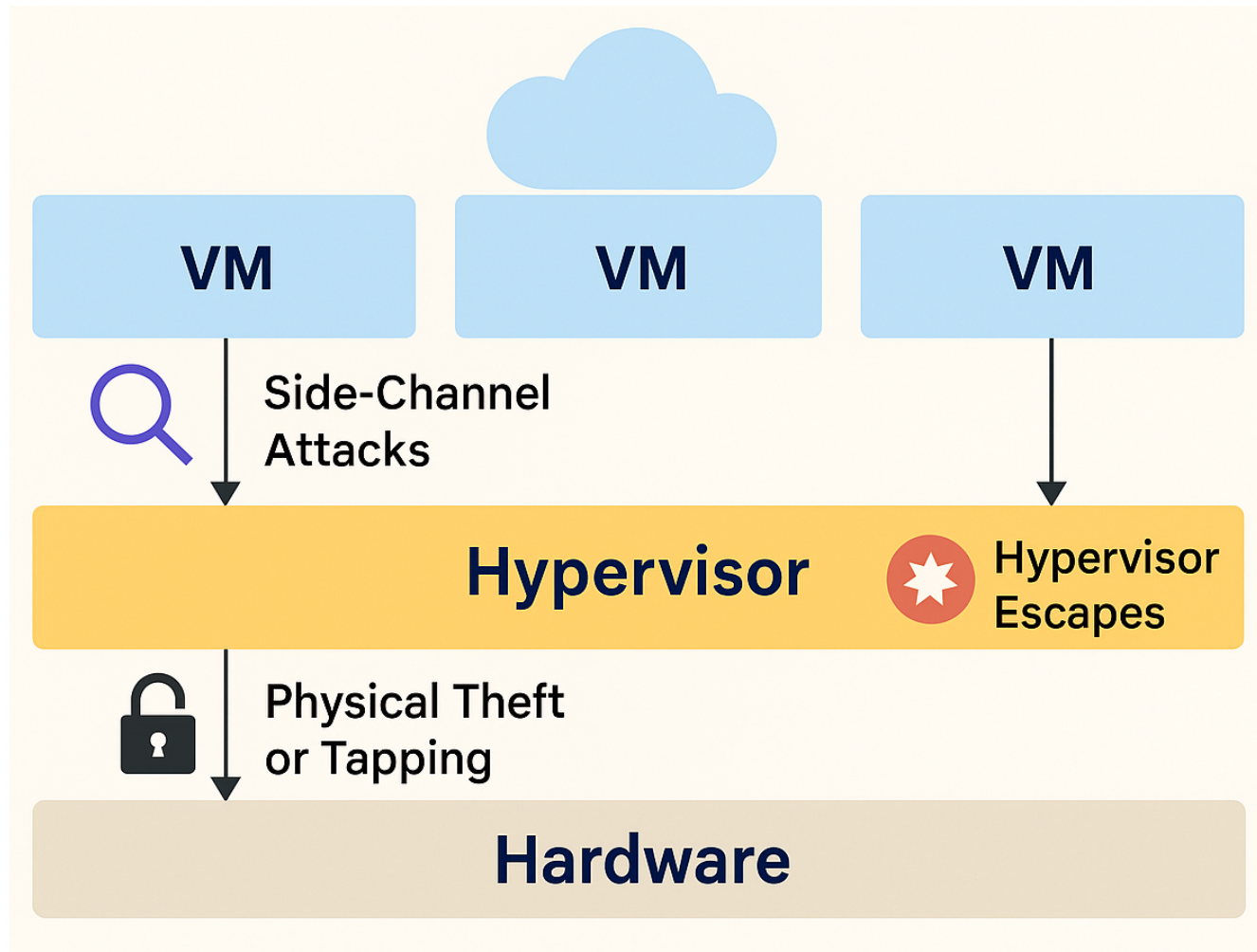
- Direct access to RAM, disk, or network cables can expose unencrypted data
- Cold boot attacks, PCIe bus sniffing

Meltdown & Spectre (2018)

- Critical microarchitectural vulnerabilities in modern CPUs that leak data via **speculative execution** and caching behavior
- Impact on Confidentiality
 - Attackers could read kernel memory or other process memory—even from sandboxed or unprivileged code
 - Broke core CPU isolation guarantees
- Relevance to the Cloud
 - Cross-VM leakage in multi-tenant environments
 - Cloud providers issued emergency patches; some disabled features like Hyper-Threading
 - Highlighted need for hardware-enforced isolation (e.g., TEEs)



How those attacks map to cloud layers?



Legal and Regulatory Risks

- Data Access via Subpoenas
 - Governments can compel cloud providers to hand over customer data—often without notifying the customer.
- Cross-Border Jurisdiction Conflicts
 - Different legal regimes (e.g., EU vs US) create uncertainty for cloud-stored data.
- The CLOUD Act (U.S.)
 - Allows U.S. authorities to access data held by U.S. companies—even if stored abroad
 - Raises concerns for foreign customers using American cloud services

Legal and Regulatory Risks: an Italian Job

“US laws had several shortcomings that impede the protection of personal data and violate the GDPR. ... US public authorities’ use of and access to EU data do not comply with the principles of proportionality...”

https://www.europarl.europa.eu/RegData/etudes/STUD/2021/694678/IPOL_STU%282021%29694678_EN.pdf

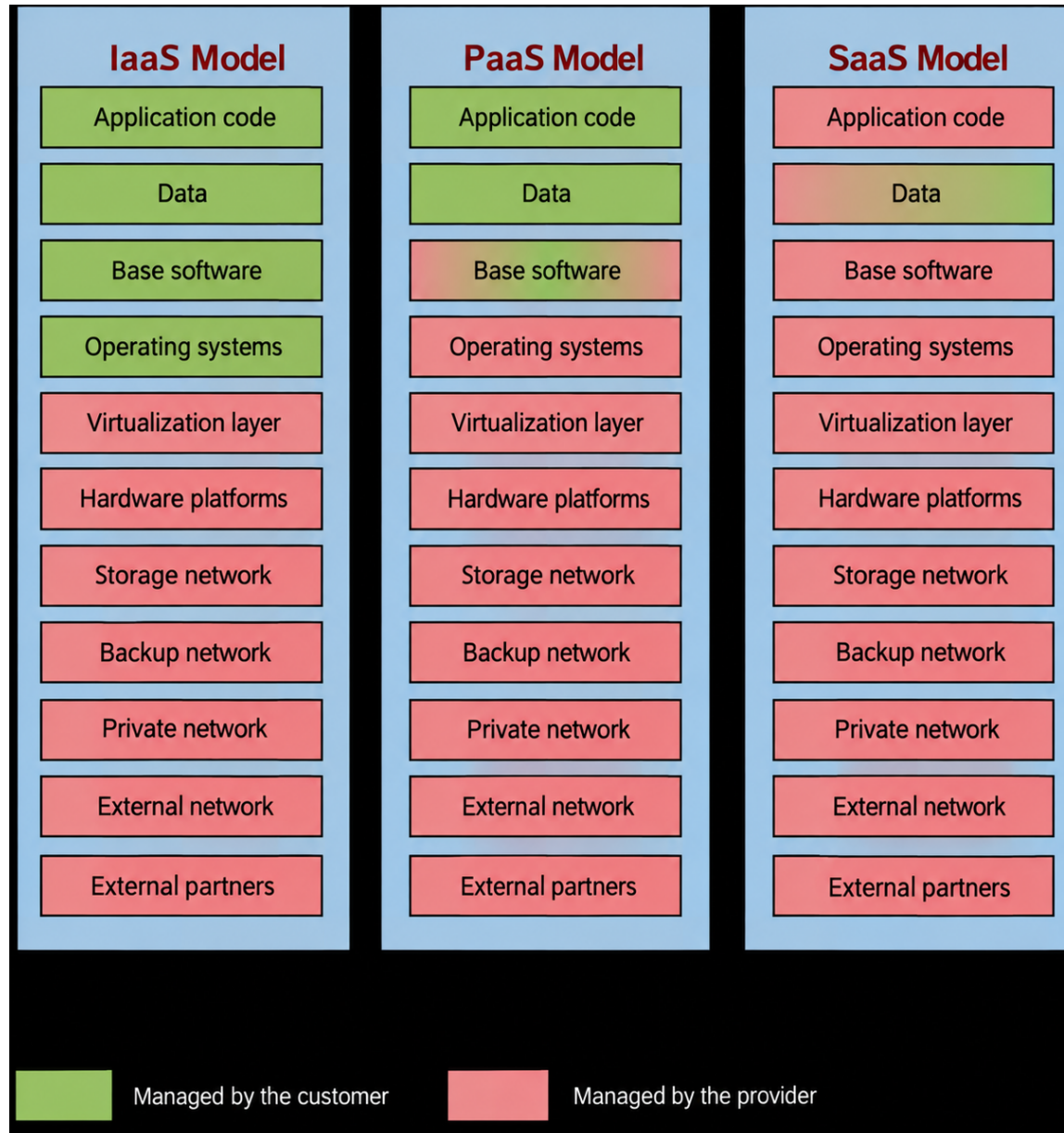
- In 2020, the European Court of Justice (ECJ) invalidated the Privacy Shield agreement between the EU and US, citing insufficient protection of EU citizens’ data from US surveillance.
- **In 2021, Italy’s Garante della Privacy warned public administrations about relying on U.S. cloud platforms for storing sensitive citizen data, especially where health or biometric data was involved.**
- Core Risk
 - Even if data is physically stored in Europe, U.S. law (like the CLOUD Act) may force cloud providers to disclose it—potentially violating GDPR.

Case Study: Capital One Breach

- Incident Summary
 - A former AWS employee exploited a misconfigured web application firewall (WAF) to access over 100 million customer records from Capital One's cloud infrastructure.
- Attack Vector
 - Gained access using **AWS EC2 metadata** service
 - No need for login credentials – exploited over-permissive IAM roles
 - Downloaded data including credit applications and Social Security numbers
- Key Risks Highlighted
 - Cloud misconfigurations can expose sensitive data
 - **Insider-level knowledge** (attacker had AWS experience) enabled lateral movement
- Traditional perimeter defenses are ineffective in cloud-native environments



IaaS, PaaS, SaaS



Cloud Service Models & Threat Exposure

Model	User controls	Provider controls	Security Exposure
IaaS	OS, runtime, apps	VM, hypervisor, hardware	VM isolation flaws, admin access, misconfigurations
PaaS	Apps, data	OS, runtime, scaling	App-layer exploits, noisy neighbors, platform trust
SaaS	Data, config	Everything else	Minimal control, full trust in provider's stack

- The more control the provider has, the greater the trust required — and the more critical confidentiality protections (like TEEs) become.

What is Data-in-Use?

- Data-at-Rest
 - Stored on disk or persistent storage (e.g., databases, SSDs)
 - Protected by encryption at rest (e.g., AES, BitLocker)
- Data-in-Transit
 - Moving across networks (e.g., HTTPS, VPNs, TLS)
 - Protected by encryption in motion (e.g., TLS, IPsec)
- **Data-in-Use**
 - Actively being processed in memory (RAM/CPU)
 - Exposed in plaintext during computation
 - Vulnerable to attacks from compromised OS, hypervisor, or insiders

Traditional encryption protects data **before** and **after** use—but not while it's being used.
This is exactly where Confidential Computing intervenes.

Why Confidential Computing Now?

- Explosive growth in cloud adoption
 - Enterprises increasingly run critical workloads in public cloud
 - Multi-tenant infrastructure heightens exposure to insider and external threats
- Shift to remote & distributed work
 - More sensitive data processed off-premise
 - Expanded attack surface across devices, networks, and cloud services
- Tighter regulations & compliance
 - GDPR, HIPAA, Schemers II, and sector-specific mandates
 - Need for verifiable control over data, even in third-party environments

Organizations need stronger guarantees that data remains protected even during processing—especially in untrusted or shared environments. Confidential Computing delivers that.

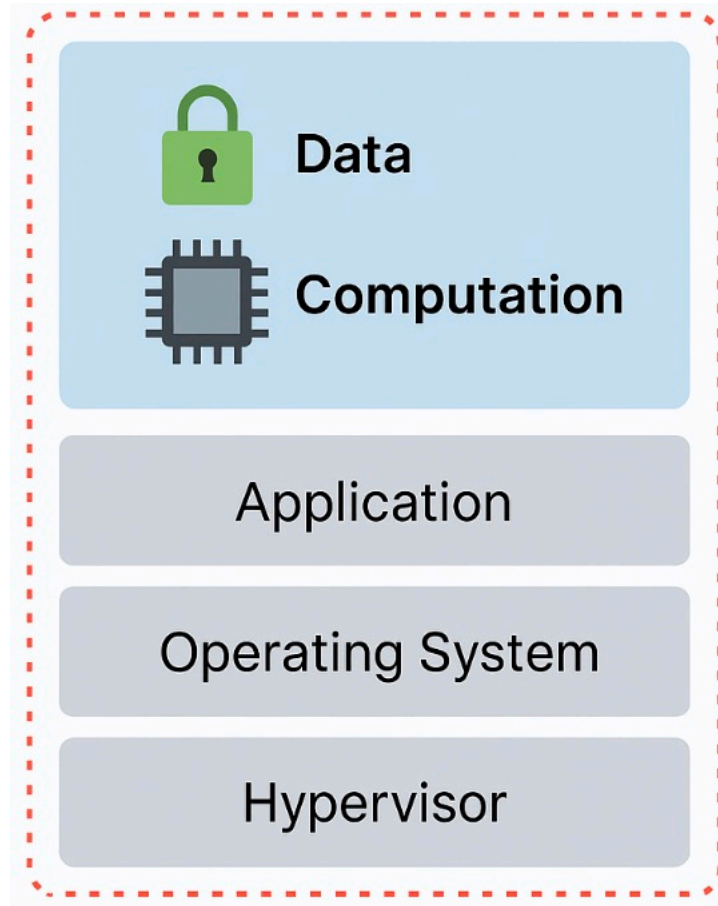
Section 2

Definitions and Goals

What is Confidential Computing?

- Confidential Computing is the practice of protecting data **while it is being used**—i.e., during computation in memory—by performing the computation in a *hardware-based* Trusted Execution Environment (**TEE**).
- Core idea
 - Data remains encrypted or isolated even during processing
 - Reduces the need to fully trust the host OS, hypervisor, or cloud provider
- Why it matters
 - Enables secure cloud computing, multi-party collaboration, and verifiable privacy in untrusted environments

Trust Boundaries





Data-in-Use Protection

- Ensuring that data remains protected even while it is being processed in memory—not just stored or transmitted
- How it works
 - Uses hardware-based Trusted Execution Environments (TEEs)
 - Data and code are isolated from the host OS, hypervisor, and other workloads
 - Memory is encrypted on the fly and decrypted only inside the CPU
- What it enables
 - Protects against insider threats and malware on the host
 - Enables secure workloads in shared or untrusted infrastructure (e.g., public cloud)

Trusted Execution Environments

- What is a TEE?
 - A hardware-protected area of a processor that ensures code and data loaded inside are isolated and confidential
- Key features
 - Isolated execution from the OS, hypervisor, and other apps
 - Memory encryption and integrity checks
 - Remote attestation to prove trust to third parties
- Real-world examples
 - Intel SGX, AMD SEV, Intel TDX, ARM TrustZone
 - Used in cloud VMs, secure mobile apps, and multi-party computation

Isolated Execution

- What Is Isolated?
 - **Code** and **data** inside a TEE (e.g., enclave or secure VM)
 - Memory regions and registers used during computation
- Isolated From Whom?
 - Operating System and Hypervisor
 - Cloud provider admins and other tenants
 - Malware or compromised kernel components
- Why It Matters
 - Even if the host is compromised, the sensitive workload stays protected
 - Enables trustworthy processing in untrusted environments

Confidentiality vs Integrity vs Attestation

-  Confidentiality
 - Prevents unauthorized access to data-in-use
 - Even privileged software (OS, hypervisor) cannot read enclave memory
-  Integrity
 - Ensures that the code and data inside the TEE have not been tampered with
 - Defends against injection, memory modification, or control-flow attacks
-  Attestation
 - Mechanism to cryptographically prove that trusted code is running inside a genuine TEE
 - Enables remote parties to verify the trustworthiness of the workload

Attestation Basics

- What Is Attestation?
 - A process that allows a remote party to verify that a specific code is running inside a genuine, untampered TEE
- How It Works
 - TEE generates a signed **quote** with **measurements** of the code and data
 - The quote is sent to the **remote verifier**
 - Verifier checks the signature and measurements against expected values
- Why It Matters
 - Enables trust in remote computation, even in untrusted infrastructure
 - Forms the basis for secure key provisioning, access control, and policy enforcement

Attestation Basics: quote

- Definition
 - A cryptographically signed report generated by a TEE that proves:
 - What code is running
 - In which environment (hardware identity, TEE version)
 - That the enclave has not been tampered with
- What's Inside a Quote?
 - Measurement: hash of enclave code and data
 - TEE identity: CPU keys, platform version
 - Optional user data: e.g., public keys or app-specific info
 - Digital signature by a trusted attestation key
- Why It's Important
 - The quote allows external verifiers to trust the enclave's state without direct access to the host system.

Attestation Basics: measurement

- Definition
 - A cryptographic hash (e.g., SHA-256) that uniquely identifies the initial code and data loaded into a Trusted Execution Environment (TEE)
- Purpose
 - Acts like a fingerprint of the enclave's trusted state
 - Ensures that the enclave hasn't been altered or tampered with
- When It's Computed
 - During enclave or secure VM initialization
 - Stored inside the attestation quote, and used for verification
- Why It is important
 - Verifiers can compare the received measurement to an expected value
 - Only known and approved code will be trusted or provisioned with secrets

Attestation Basics: remote verifier

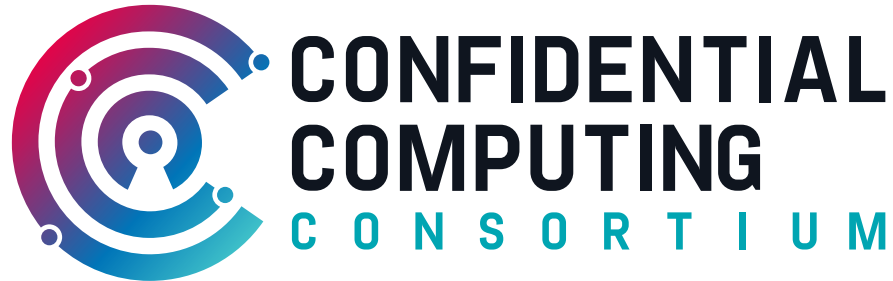
- Definition
 - A trusted external party that receives an attestation quote from a TEE and validates its authenticity and trustworthiness
- Typical Roles
 - A cloud customer, validating that their app is running securely on a provider's TEE
 - A key management system (KMS), checking enclave integrity before releasing secrets
 - A third-party auditor, ensuring compliance or policy enforcement
- What the Verifier Checks
 - Is the quote signed by a genuine TEE?
 - Does the measurement match the expected code?
 - Is the TEE version/trust level acceptable?
- Why It is Important
 - Enables trust without direct control over the platform—a cornerstone of confidential computing

Example: Azure Remote Verifier

- Azure Attestation Service (MAS / MAA)
 - A managed service by Microsoft that verifies the integrity of Trusted Execution Environments like Intel SGX or AMD SEV
- Supports **Open Enclave**, VBS (virtualised) enclaves, and TPMs
- Features:
 - Accepts enclave quotes and returns attestation tokens
 - Can enforce **policy validation** (e.g., only allow trusted enclave measurements)
 - Used for **key release**, secure API access, and multi-tenant services
- <https://learn.microsoft.com/en-us/azure/attestation/>

Example: Intel Attestation Service

- Intel's official attestation backend for Intel SGX enclaves
- Validates quotes issued by SGX-capable CPUs
- Produces a signed attestation report that confirms the enclave's measurement, platform, and SGX version
- Typically used by cloud services, edge devices, or Open Enclave SDK-based apps
- **Key feature:** Intel holds the root key, ensuring that only genuine SGX hardware can produce valid quotes
- <https://api.portal.trustedservices.intel.com/>



- A Linux Foundation project formed in 2019 to promote open collaboration and standards for confidential computing technologies
- Founding members: Intel, Microsoft, Google, Red Hat, Arm, AMD
 - Dozens of contributors from hardware, cloud, and security communities
- What It Does
 - Develops open-source projects (e.g., Open Enclave, Enarx, Keystone)
 - Promotes interoperability and shared terminology
 - Drives adoption of TEEs across cloud, edge, and enterprise use cases
- Why It Matters
 - Aligns industry efforts to make data-in-use protection widely available and vendor-neutral



Goals of Confidential Computing

- Protect workloads from cloud operators
 - Ensure that even cloud administrators and hypervisors cannot inspect or tamper with sensitive data during execution
 - Enable secure processing in untrusted or shared environments
- Enable multi-party computation
 - Multiple parties can contribute private data to a joint computation without revealing it to each other
 - Use cases: federated learning, collaborative analytics, secure auctions
- Support verifiable computation
 - Allow remote parties to verify that a specific workload ran inside a genuine, untampered TEE
 - Enables trust and compliance in outsourced computing

Applications Enabled

- Finance
 - Secure processing of sensitive transactions, credit scoring, and risk models in untrusted environments
 - Enables private market analytics and regulatory compliance (e.g., GDPR, PSD2)
- Healthcare
 - Confidential sharing of medical records across hospitals for joint diagnosis or research
 - Enables privacy-preserving clinical trials and genomic data analysis
- **Machine Learning**
 - Run models on private or proprietary data without exposing inputs or IP
 - Supports confidential inference and federated learning with strong guarantees

Example: Confidential ML

- The Problem
 - A company wants to run a proprietary ML model on private customer data
 - The model is deployed in a public cloud → risk of data leakage or IP theft
- Solution with Confidential Computing:
 - Deploy the model **inside** a Trusted Execution Environment (TEE)
 - Data is decrypted only inside the TEE, never exposed to the OS, admin, or cloud provider
 - Model weights and code remain sealed from outsiders
- Benefits
 - Enables **privacy-preserving** inference
 - Protects both model IP and user inputs
 - Enables regulated industries (e.g., health, finance) to use ML securely in the cloud

What Confidential Computing Does Not Solve

- App bugs:
 - TEEs do not protect against logic errors, memory corruption, or unsafe coding practices inside the enclave
- Malicious inputs / Data poisoning
 - Confidential Computing assumes trusted inputs
 - Garbage in → garbage out still applies
- Side-Effects Outside the TEE
 - Once data leaves the enclave (e.g., for storage or output), it's vulnerable again
 - Requires secure I/O and storage integration
- Developer Trust Assumptions
 - TEEs protect execution, not design intent or developer honesty

Confidential Computing reduces the trusted computing base.
But, it doesn't eliminate the need for secure software engineering.

Section 3

Threat Models and Attacker Capabilities

What is a Threat Model?

- Definition
 - A threat model defines what assets need protection, who the attackers are, and what capabilities they have
- Key Components
 - **Assets** – What needs to be protected (e.g., data, code, secrets)
 - **Adversaries** – Who might try to compromise the system (e.g., insiders, hypervisors)
 - **Assumptions** – What components are trusted or untrusted
 - **Attack Vectors** – How attackers might succeed
- Why It Matters
 - Guides system design and security architecture
 - Helps clarify the limits of what a technology like Confidential Computing can and cannot defend against

Example Threat Model: Cloud-Based ML Inference API

- Scenario:
 - A company deploys a machine learning model in the public cloud to process sensitive customer data via an API (e.g., loan approvals, medical predictions).
- Assets to Protect
 - User input data (e.g., health or financial information)
 - ML model parameters (intellectual property)
 - Inference results

Example Threat Model: Cloud-Based ML Inference API

- **Adversaries:**
 - Cloud provider insiders (admins with root access)
 - Other tenants on the same host
 - Remote attackers exploiting vulnerabilities
 - Malicious OS or hypervisor
- **Trust Assumptions**
 - Trust the TEE hardware (e.g., Intel SGX, AMD SEV)
 - Distrust the cloud OS, hypervisor, network stack
 - Trust the client and verifier (e.g., quote validation endpoint)

Example Threat Model: Cloud-Based ML Inference API

- Security goals (what to prevent):
 - Leakage of input or model outside the TEE
 - Tampering with code or data during inference
 - Unauthorized access to enclave memory
 - Running inference in an untrusted or modified environment
- Out-of-Scope threats:
 - Bugs in the ML model code (e.g., buffer overflows, logic errors)
 - Malicious user inputs (e.g., adversarial examples, model evasion)
 - Side-channel attacks (unless specifically mitigated)
 - Denial-of-Service (DoS) attacks on the enclave or API

TEE Threat Model Overview

- Trusted Components:
 - CPU / TEE hardware (e.g., Intel SGX, AMD SEV, ARM TrustZone)
 - Enclave code and data (if loaded from a known, measured state)
 - Remote verifier and cryptographic attestation keys
- Untrusted Components:
 - Operating System (OS) and hypervisor
 - Cloud provider administrators
 - Other tenants or VMs on the same host
 - Host firmware (in many threat models)
- Goal
 - Protect sensitive computation even if everything outside the TEE is compromised

Typical Confidential Computing Threat Model

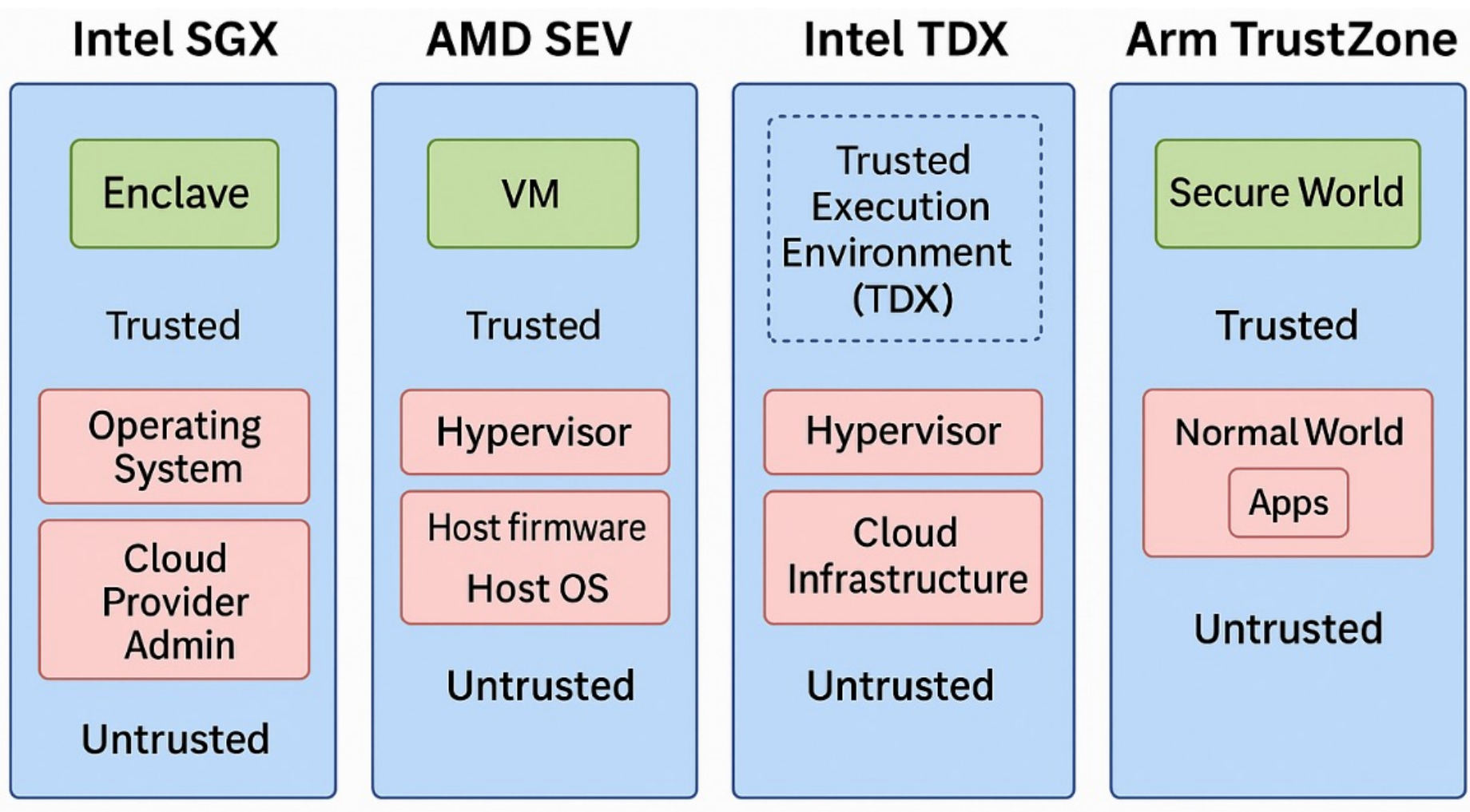
- Assumptions
 - The TEE hardware and attestation keys are trusted
 - The application inside the enclave is correct and unmodified
 - The remote verifier can validate TEE identity and measurements
- Adversaries (Untrusted)
 - OS, hypervisor, cloud platform
 - Cloud provider insiders and other tenants
 - Physical attackers (to some extent, depending on TEE)
- Security Guarantees
 - Confidentiality and integrity of code and data inside the enclave
 - Remote attestation ensures the enclave is genuine and untampered
 - Secrets remain inaccessible outside the TEE—even to root users

Threat models vary across TEE technologies (SGX, SEV, TDX), and may exclude certain physical or side-channel attacks.

TEE Threat Model Comparison

Aspect	Intel SGX	AMD SEV-SNP	Intel TDX	TrustZone
Granularity	App-level enclaves	Full VM	Full VM	Secure world (system-wide)
Trusted	CPU, enclave code	CPU, firmware, VM guest	CPU, TDX modules, guest OS	CPU, secure OS
Untrusted	OS, hypervisor, cloud admin	Hypervisor, host OS, admin	Hypervisor, cloud infra	Normal world (OS + apps)
Attestation	Intel IAS	Built-in, remote cert chain	SEV+ guest report	Vendor-specific
Mem. Protector	Enclave memory EPC	Full guest mem. encrypt.	Full guest mem. encrypt.	No
Use-cases	Fine-grained isolation	Secure cloud VMs	Secure cloud VMs	Mobile, IoT, edge

TEE Threat Model Comparison



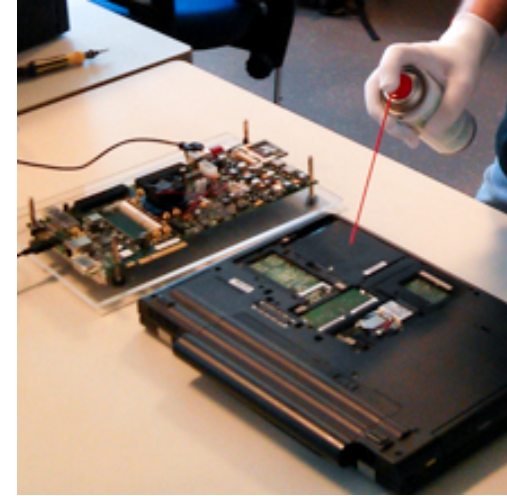
Physical Access Attacks

- **Memory Dump Attacks**
 - Attackers with physical access can dump RAM contents using specialized tools
 - Risk: sensitive data (e.g., keys, session tokens) may still reside in memory
- **Cold Boot Attacks**
 - Exploit the fact that RAM retains data briefly after power loss
 - By quickly transferring the memory chips to another system, an attacker can recover encryption keys or private data
- **Why It Matters**
 - In cloud or data center scenarios, a rogue employee or attacker with hardware access could compromise running workloads
 - TEEs protect memory regions with encryption and access control, mitigating these risks

Cold Boot Attack Timeline



Let's We Remember: Cold Boot Attacks on Encryption Keys



-  Step-by-step attack flow:
 1. System is Running
 - Sensitive data (e.g., encryption keys, passwords) is loaded in RAM during normal operation
 2. Forced shutdown
 - Attacker powers off the machine abruptly (e.g., by cutting power or removing battery)
 3. Quick RAM extraction
 - RAM chips are quickly removed and cooled (e.g., with freeze spray or liquid air)
 - Cooling extends the data retention window
 4. RAM transfer
 - Chips are inserted into a controlled analysis system
 - Booted into a lightweight OS that immediately dumps memory contents
 5. Data recovery
 - Tools extract keys, credentials, or sensitive application data from the memory image

OS-level Threats

- Malicious or Compromised Kernel Modules
 - Kernel modules can intercept, modify, or log system behavior
 - In a cloud VM, a privileged attacker can load **rogue** modules to access process memory or I/O
 - Risk: sensitive app data or keys are exposed to the OS
- System Call (Syscall) Abuse
 - TEEs still rely on the OS for I/O (e.g., file access, networking)
 - A malicious OS can tamper with syscall responses, leak metadata, or cause denial-of-service
 - e.g., falsified file contents or replayed network data

TEEs isolate sensitive code/data from the OS kernel. Even with root-level compromise, enclave memory and execution remain protected.

Fork Bombs

- A denial-of-service (DoS) attack that rapidly creates new processes to exhaust system resources (CPU, memory, process table)
- Exploits the `fork()` system call to spawn copies of itself recursively
- Quickly overwhelms the process scheduler, leading to system freeze or crash

```
:(){ :|:& };:
# Classic Bash fork bomb – don't run it!
```

**DONT
DO THIS
AT HOME**

- Though it doesn't exfiltrate data, a fork bomb can:
 - Disrupt TEE execution by starving resources
 - Trigger reboots, affecting system availability
 - Bypass defenses if resource limits (e.g., `ulimit`) are not enforced

SGX Enclaves prevents fork bombs by design - `fork()` sys-call not supported [as `exec()`, `clone()`]

Cloud Admin Threats

-  VM Snapshots
 - Cloud admins can create live snapshots of running virtual machines
 - Risk: memory, disk, and CPU state—including sensitive keys or credentials—may be captured
 - Snapshots can be cloned, paused, or inspected offline
-  Disk Access
 - Admins may access virtual disk images stored in cloud storage
 - Data-at-rest encryption helps, but decryption keys are often **in memory** during runtime
-  Why It Matters
 - Traditional VMs trust the cloud admin implicitly
 - Confidential Computing ensures that even privileged infrastructure cannot access sensitive data
 - Memory encryption (SEV,TDX)
 - Isolated enclaves (SGX)

Hypervisor Attacks

- VM Introspection (VMI)
 - Hypervisors can inspect guest VM memory and CPU state without detection
 - Intended for security or debugging, but can be abused to extract secrets or monitor activity
 - e.g., scanning memory for SSH keys or database records
- Side-Channel Surfaces
 - Shared hardware (e.g., caches, TLBs, branch predictors) opens the door to microarchitectural side-channel attacks
 - A compromised hypervisor or co-located VM can **leak enclave** or VM data
 - e.g., Spectre, LITF, Prime+Probe
-  How TEEs Help
 - Confidential VMs (SEV, TDX) encrypt memory to block introspection
 - Enclaves (SGX) prevent even the hypervisor from reading code/data

Google these

Side-Channel attacks to Enclaves

- Enclave memory (EPC) is encrypted
- Side-channel information and control over system resources remain attack surfaces.
- Usable techniques from a compromised hypervisor:
 - Page Table Manipulation (Page-Fault Side Channel / Controlled-Channel Attack)
 - By monitoring which pages are accessed when, an attacker can infer control flow or data-dependent behavior.
 - Cache Timing Attacks
 - shared CPU caches are not isolated, can be leveraged to infer memory access patterns
 - Branch Prediction & Speculative Execution Attacks
 - Sgxpectre [S&P'19], BranchScope, and SMOtherSpectre
 - And more: Interface-Based Attacks, interrupt and Scheduling Attacks ..

Bottom line: ENCLAVES are not invincible

Side-Channels: An Overview

- What are side-channels ?
 - Side-channels exploit indirect signals (e.g., timing, power, memory access patterns) to leak secret information from secure computations — even if direct access is blocked.
- Common Side-Channel Types:
 - Cache Timing (e.g., Prime+Probe, Flush+Reload)
 - Exploits shared CPU cache to detect memory access patterns
 - Reveals control flow or secret-dependent table lookups
 - Branch Prediction:
 - Leaks information via mispredicted branches and speculative execution
 - DRAM-Based (e.g., Rowhammer)
 - Uses memory access timing or bit flipping to infer or manipulate data
 - Can cross VM or enclave boundaries in shared hardware
- Side-channels exploit what TEEs don't encrypt: hardware behavior. mitigating them requires **isolation**, **obfuscation**, and **system-level defenses**.

Out-of-Scope Threats

- In this course, we will not cover or deal with the following threats:
 - developer bugs:
 - TEEs do not protect against logic flaws, unsafe memory access, or vulnerable enclave code
 - Example: buffer overflows, use-after-free, insecure crypto
 - Supply Chain Attacks
 - If the enclave code is compromised before loading (e.g., during build or signing), the TEE will still run it
 - TEEs verify integrity, not correctness or intent
 - Denial-of-Service (DoS)
 - A malicious OS or hypervisor can starve, pause, or crash enclaves and VMs
 - TEEs do not guarantee availability, only confidentiality and integrity

Confidential Computing narrows the Trusted Computing Base—but doesn't eliminate all risks.

Realistic Threat Modeling in Practice

- Believing that TEEs (or any secure hardware) provide complete, foolproof isolation from all attackers and attack vectors **is simply wrong**.
- TEEs reduce the trusted computing base, but
 - Still rely on **complex microarchitecture** (vulnerable to side-channels)
 - Need secure enclave code — **bugs** can still leak secrets
 - Are impacted by **hardware/firmware vulnerabilities** (e.g., Foreshadow, SgxPectre)
- Good threat modeling is:
 - **Specific**: Clearly states what's trusted/untrusted
 - **Pragmatic**: Considers real-world limitations
 - **Evolving**: Adapts as new attacks and defenses emerge

Adversarial Thinking Exercise

[attack]

- Put yourself in the attacker's shoes...
- You are a *malicious cloud provider admin*. A customer is running a sensitive workload in a TEE (e.g., Intel SGX enclave). You cannot directly read the enclave's memory.
- What could you try?
 - Can you observe page access patterns or trigger page faults?
 - Can you use side-channel timing (e.g., cache, branch prediction)?
 - Can you tamper with the data returned from syscalls or OCALLs?
 - Can you interfere with attestation or enclave startup?
 - Can you deny service to disrupt availability?

Adversarial Thinking Exercise

[defense]

- Now try to mitigate those attacks
- *How do you mitigate those attacks?*

Adversarial Thinking Exercise

[attack]

- **Can you observe page access patterns or trigger page faults?**
- Yes, especially with SGX and similar TEEs that delegate page table management to the untrusted OS.
 - Attackers can evict pages and observe when the enclave accesses them again.
 - This leads to controlled-channel attacks and page-fault side-channel attacks.
- *Mitigation: data-oblivious programming, ORAM techniques, T-SGX.*

Adversarial Thinking Exercise

[attack]

- **Can you use side-channel timing (e.g., cache, branch prediction)?**
- Yes, very often.
 - Shared CPU resources like L1/L2 caches, branch predictors, and DRAM channels can leak information through timing differences.
 - Techniques include Prime+Probe, Flush+Reload, Spectre-SGX, and SMOtherSpectre.
- *Mitigation: constant-time programming, cache partitioning (CAT), microcode patches, fencing/barriers.*

Adversarial Thinking Exercise

[attack]

- **Can you tamper with the data returned from syscalls or OCALLs?**
- Yes.
 - EEs rely on untrusted OS code for I/O, networking, and other services.
- A malicious OS can manipulate OCALL results, delay responses, or inject malformed data.
- Mitigation: verify OCALL responses inside the enclave, use authenticated channels or sealed I/O.

in a later chapter

Adversarial Thinking Exercise

[attack]

- **Can you interfere with attestation or enclave startup?**
- **Partially, yes - but not undetectably.**
 - The OS can refuse to start an enclave, block communication, or drop attestation messages.
 - However, it cannot forge a valid attestation quote (as it's signed by hardware-backed keys).
- **Mitigation: retries, timeout logic, fallback strategies — but note: availability is not guaranteed by TEEs.**

Adversarial Thinking Exercise

[attack]

- **Can you deny service to disrupt availability?**
- Yes.
 - The OS or hypervisor can pause, terminate, or throttle enclaves and VMs at will.
 - This is a classic **out-of-scope** threat in the TEE model.
- Mitigation: retries, timeout logic, fallback strategies — but note: availability is not guaranteed by TEEs.

Trade-offs in the Threat Model

- TEEs introduce encryption, memory isolation, and syscall marshalling
 - SGX: limited EPC size → page swapping penalties
 - SEV/TDX: encryption adds latency and CPU overhead
- Programmability Constraints:
 - Restricted features: no fork(), limited threading, no system calls inside enclaves
 - Developers must re-architect apps to:
 - avoid dynamic memory patterns
 - Split logic across enclave/non-enclave code
- User Experience (UX) Challenges:
 - Remote attestation adds complex trust bootstrapping
 - Debugging is harder (encrypted state, limited introspection)
 - Limited visibility into failures from inside the enclave

Use Cases to Threat Models (1/3)

- ML Inference
 - Goal: Protect sensitive input data and proprietary model
- Threats:
 - Cloud OS/admin snooping
 - Side-channel leakage
- TEE Strategy:
 - Run model inside enclave
 - Use attestation before sending inputs
 - Minimize memory footprint to reduce leakage

Use Cases to Threat Models (2/3)

- Databases (e.g., Encrypted SQL)
- Goal: Process encrypted queries on sensitive data
- Threats:
 - Cloud insiders
 - Query pattern leakage
- TEE Strategy:
 - Deploy DB engine inside enclave
 - Encrypt I/O and metadata
 - Obfuscate access patterns where possible

Use Cases to Threat Models (3/3)

- Key storage/management
- Goal: Store and use cryptographic keys securely in untrusted environments
- Threats:
 - Disk theft, memory scraping, admin access
- TEE Strategy:
 - Seal keys to enclave
 - Release keys only after successful remote attestation
 - Use monotonic counters / nonces to prevent replay

Different use cases demand different trust assumptions and mitigation strategies:
no one-size-fits-all model.

Summary

- Motivation: cloud threats are real
- Confidential computing = protect data-in-use
- Threat models determine system design

Starters for Q&A

- What do you *not* trust in a public cloud?
- Which attacker seems most dangerous?